Рассуждающие языковые модели с нуля

Учебное пособие для начинающих
по материалам книги Себастьяна Рашки
«Build a Reasoning Model (From Scratch)»
и репозитория github.com/rasbt/reasoning-from-scratch

Адаптировано для чтения без предварительной подготовки.
После каждой главы — 5 вопросов для самопроверки.

Как читать это пособие

Это пособие — упрощённый пересказ книги Себастьяна Рашки «Build a Reasoning Model (From Scratch)» («Создай рассуждающую модель с нуля») и сопровождающего её репозитория на GitHub. Книга учит одной вещи: **как обычную языковую модель (LLM) превратить в «рассуждающую» модель** — такую, которая перед ответом «думает», расписывает ход решения и за счёт этого решает сложные задачи (например, математические) гораздо лучше.

Оригинал рассчитан на программистов и содержит много кода на Python. Здесь код почти убран: мы сосредоточимся на **идеях и механике** — чтобы после прочтения вы понимали, что происходит и почему это работает, даже если вы не пишете код.

Структура пособия повторяет книгу:

- **Глава 0** (добавлена специально для вас) — необходимый минимум: что такое LLM, токены, вероятности. Без неё остальное будет непонятно.
- **Глава 1** — что такое «рассуждение» у моделей и какие есть пути его добавить.
- **Главы 2–3** — подготовка: как модель генерирует текст и как измерять её «ум».
- **Главы 4–5** — первый путь: улучшаем рассуждения без дообучения, прямо в момент ответа.
- **Главы 6–7** — второй путь: дообучаем модель рассуждать с помощью обучения с подкреплением (так сделан DeepSeek-R1).
- **Глава 8** — третий путь: дистилляция — маленькая модель учится у большой.

Правила работы: читайте главы по порядку. В конце каждой главы — блок из 5 вопросов. Отвечайте своими словами (письменно) и присылайте ответы мне в чат в формате «Глава N: 1) ... 2) ... 3) ... 4) ... 5) ...». Я проверю, объясню ошибки и скажу, можно ли идти дальше.

Обозначения в тексте:
Жёлтые блоки «Простыми словами» — то же самое, но совсем просто.
Зелёные блоки «Аналогия» — бытовое сравнение, чтобы идея «легла».
Синие блоки — определения терминов (их стоит запомнить).
Фиолетовые блоки — ключевая мысль главы.
Красные блоки — вопросы для самопроверки.

Глава 0. Необходимый минимум: как вообще устроена LLM

Эту главу в книге заменяет предполагаемый опыт читателя. Нам она нужна, чтобы дальше всё было понятно. Здесь нет ничего сложного — только четыре идеи.

0.1 Модель — это «угадыватель следующего слова»

Большая языковая модель (LLM, Large Language Model) — это компьютерная программа, которую обучили на огромном количестве текстов из интернета, книг и статей. Её единственное базовое умение звучит скромно: **по началу текста предсказать, какое слово (точнее, кусочек слова) идёт дальше**.

Например, видя текст «Столица Франции — », модель предсказывает, что дальше почти наверняка идёт «Париж». Повторяя это предсказание много раз подряд (предсказали слово → приклеили его к тексту → предсказали следующее), модель пишет целые статьи, письма и решения задач.

Аналогия. Представьте человека, который прочитал всю библиотеку мира и теперь играет в игру «продолжи фразу». Он не хранит тексты наизусть — он уловил закономерности: после каких слов какие обычно идут. LLM — такой «игрок», только закономерности у него записаны числами.

Параметры (веса) — числа внутри модели, в которых закодированы все выученные закономерности. У модели из этой книги их 600 миллионов (0.6 миллиарда, «0.6B»). У гигантов вроде DeepSeek-R1 — 671 миллиард. Чем больше параметров, тем «умнее» модель, но тем больше ей нужно памяти и вычислений.

Обучение (training) — процесс подбора этих чисел на примерах. **Инференс (inference)** — использование уже обученной модели для генерации ответов. Обучение дорогое и долгое, инференс — сравнительно дешёвый. Эти два слова будут встречаться постоянно.

0.2 Токены: модель читает не буквы и не слова

Прежде чем текст попадёт в модель, его режут на **токены** — куски текста. Токен может быть целым словом («кот»), частью слова («про», «грамм», «ист») или знаком препинания. У каждого токена есть номер. Модель работает только с номерами.

Токенизатор — программа-переводчик: текст → список номеров токенов (это называется **кодирование**) и обратно: номера → текст (**декодирование**). **Словарь** токенизатора — полный список всех токенов, которые он знает. У модели из книги словарь содержит 151 936 токенов.

Аналогия. Токенизатор — как азбука Морзе: перед отправкой сообщение переводят в код, после получения — обратно в текст. Сама «линия связи» (модель) понимает только код.

0.3 Вероятности: модель не «знает», а «оценивает шансы»

На каждом шаге модель выдаёт не одно слово, а **оценку для каждого из 151 936 токенов словаря**: насколько он подходит в качестве следующего. Эти «сырые» оценки называются **логитами** (logits). Их можно превратить в **вероятности** — числа от 0 до 1, которые в сумме дают 1. Например: «Париж» — 92%, «город» — 3%, «Лион» — 1%, остальное — крохи.

Дальше есть выбор: всегда брать самый вероятный токен (это называется **жадная генерация**, greedy decoding — ответ всегда одинаковый) или «подбрасывать кубик» и выбирать случайно с учётом вероятностей (**сэмплирование** — ответы каждый раз разные). Этот выбор станет важным в главе 4.

0.4 Этапы жизни модели

Современные модели готовят в несколько этапов:

1. **Предобучение (pre-training).** Модель читает триллионы слов и учится предсказывать следующий токен. Результат — **базовая модель** (base model): эрудированная, но «невоспитанная» — она просто продолжает текст, а не ведёт диалог.
2. **Пост-обучение (post-training).** Базовую модель дообучают: учат следовать инструкциям, вежливо отвечать, а в нашем случае — **рассуждать**. Вся эта книга — про пост-обучение (и про приёмы, которые работают вообще без обучения).

Ключевая мысль главы 0. LLM — это предсказатель следующего токена. Текст режется на токены, модель оценивает вероятности всех возможных продолжений, из них выбирается одно, и так по кругу. «Базовая» модель умеет только продолжать текст — всё интересное (диалог, рассуждения) добавляется потом.

Вопросы к главе 0

1. Что делает языковая модель на каждом отдельном шаге генерации текста? Как из этого получается целый связный ответ?
2. Что такое токен и зачем нужен токенизатор? Почему нельзя просто «скормить» модели буквы?
3. Чем отличается обучение (training) от инференса (inference)?
4. Чем отличается жадная генерация от сэмплирования? В каком случае модель на один и тот же вопрос всегда отвечает одинаково?
5. Что такое базовая модель и почему её обычно дообучают, прежде чем давать пользователям?

Глава 1. Что такое рассуждающие модели

Эта глава в книге — без кода, она задаёт общую картину. Разберём её подробно, потому что все остальные главы опираются на введённые здесь понятия.

1.1 Что значит «рассуждение» у LLM

В контексте LLM **рассуждение (reasoning)** — это способность модели перед финальным ответом выдавать **промежуточные шаги** решения. Вместо того чтобы сразу выпалить «ответ: 42», рассуждающая модель пишет: «Сначала найдём X... значит Y... проверим... следовательно, ответ 42».

Такую цепочку промежуточных шагов называют **цепочкой рассуждений** (chain of thought, CoT). Важно понимать: это не магическое «мышление», как у человека. Модель по-прежнему просто предсказывает следующий токен. Но когда она «проговаривает» решение по шагам, каждый следующий шаг опирается на уже написанные — и итоговая точность на сложных задачах резко растёт.

Аналогия. Попробуйте перемножить 47 × 83 в уме мгновенно — почти наверняка ошибётесь. А теперь на бумаге, столбиком, по шагам — получится. Модель «на бумаге» (в виде текста) делает то же самое: промежуточные записи разгружают «голову» и позволяют решать то, что «в один присест» не решается.

> **Психологическая параллель** (её часто приводят): у человека есть быстрое интуитивное мышление («система 1» — мгновенно узнать лицо) и медленное аналитическое («система 2» — решить уравнение). Обычная LLM отвечает «системой 1» — сразу. Рассуждающая модель имитирует «систему 2» — думает дольше, отвечает точнее.

Примеры рассуждающих моделей: DeepSeek-R1, GPT-5 Thinking, o1/o3 от OpenAI, Qwen3 в режиме «thinking». Все они перед ответом генерируют длинный «мыслительный» текст (иногда скрытый от пользователя).

1.2 Цена рассуждений

За всё надо платить. Рассуждающая модель генерирует в разы больше токенов на каждый ответ, а значит:

- отвечает **медленнее** (в главе 3 увидим: оценка рассуждающей модели заняла 182 минуты против 10 минут у обычной);
- стоит **дороже** (больше вычислений на каждый вопрос);
- для простых вопросов это **избыточно** — спросить «сколько будет 2+2» у рассуждающей модели — как вызывать аудитора, чтобы проверить чек из магазина.

Поэтому цель — не «всегда рассуждать», а уметь включать рассуждение там, где оно нужно.

1.3 Три пути к рассуждающей модели

Книга устроена вокруг трёх подходов, которые можно комбинировать. Запомните эту карту — она будет повторяться в каждой главе:

Путь	Суть	Нужно ли дообучать модель?	Главы
1. Масштабирование на инференсе (inference-time scaling)	Тратим больше вычислений в момент ответа: просим думать по шагам, генерируем несколько вариантов и выбираем лучший, просим модель улучшить свой же ответ	Нет — модель не меняется	4, 5
2. Обучение с подкреплением (reinforcement learning, RL)	Дообучаем модель: она решает задачи, за правильные ответы получает «награду» и постепенно сама учится рассуждать	Да	6, 7
3. Дистилляция (distillation)	Берём большую умную рассуждающую модель («учителя») и обучаем маленькую модель («ученика») подражать её ответам	Да (ученика)	8

Простыми словами: путь 1 — «выжать больше из того, что есть»; путь 2 — «натренировать через практику и оценки»; путь 3 — «списать у отличника, да так хорошо, что сам научишься».

1.4 Как устроен практикум книги

Автор принципиально не использует готовые «чёрные ящики»: всё строится **с нуля** на чистом PyTorch (библиотека для машинного обучения на Python), чтобы была

видна каждая деталь. За основу берётся готовая открытая модель **Qwen3 0.6B** (базовая версия, 600 млн параметров — достаточно маленькая, чтобы работать на обычном домашнем компьютере), и на неё по шагам «надстраиваются» рассуждательные способности. Для сравнения используется официальная рассуждающая версия той же Qwen3 — как ориентир, к которому мы стремимся.

> **Ключевая мысль главы 1.** Рассуждение у LLM — это генерация промежуточных шагов перед ответом, и оно резко повышает точность на сложных задачах ценой скорости и стоимости. Добавить его можно тремя путями: без обучения (хитрости на инференсе), обучением с подкреплением и дистилляцией.

Вопросы к главе 1

1. Что такое «рассуждение» применительно к LLM? Что такое цепочка рассуждений (chain of thought)?
2. Почему проговаривание промежуточных шагов повышает точность модели на сложных задачах? Приведите аналогию из жизни.
3. Назовите три пути создания рассуждающей модели и объясните главное отличие первого пути от двух других.
4. Какие минусы есть у рассуждающих моделей? Почему не стоит «рассуждать» над каждым вопросом?
5. Какая модель используется в книге как основа и почему выбрана именно маленькая модель?

Глава 2. Генерация текста предобученной моделью

Это «настроечная» глава: в ней автор поднимает рабочее окружение, загружает модель и пишет функцию генерации текста, которой будут пользоваться все остальные главы. Нам важно понять механику.

2.1 Что загружаем

Загружаются две вещи:

1. **Токенизатор** Qwen3 — переводчик «текст ↔ номера токенов» (словарь ≈ 151 тысяча токенов).
2. **Веса модели** Qwen3 0.6B — файл размером около 1,5 ГБ со всеми 600 миллионами выученных чисел.

Автор написал код модели сам, на чистом PyTorch, но загружает в него **официальные предобученные веса** Qwen3. То есть «скелет» самодельный и прозрачный, а «знания» — настоящие, натренированные компанией Alibaba на триллионах токенов. Понимать внутренности архитектуры для книги не обязательно: мы не меняем модель изнутри, а надстраиваем рассуждения сверху.

Код запускается даже на обычном процессоре (CPU); видеокарта (GPU) ускоряет работу и становится практически необходимой только в главах 5–7, где много вычислений.

2.2 Как рождается текст: пошаговый цикл

Генерация устроена как цикл, и его стоит понять до автоматизма:

1. Текст запроса кодируется в токены: «Объясни, что такое LLM» → [15234, 871, ...].
2. Токены подаются в модель. Модель выдаёт логиты — оценки для всех 151 936 возможных следующих токенов.
3. Выбирается один токен (пока — жадно: самый вероятный).
4. Выбранный токен приклеивается к последовательности, и всё повторяется со 2-го шага.
5. Цикл останавливается, когда модель выдала специальный токен конца текста или достигнут лимит длины.

Токен конца последовательности (EOS, end-of-sequence) — специальный служебный токен, например `<|endoftext|>`. При обучении им разделяли разные документы. Если не останавливать генерацию на этом токене, модель «перескочит» на новый случайный текст, и ответ превратится в бессмыслицу. Поэтому функция генерации всегда проверяет: выпал EOS — стоп.

Аналогия. Модель пишет, как человек печатает СМС с подсказками клавиатуры, нажимая каждый раз на первое предложенное слово. Одно слово за раз, никакого «плана всего ответа» заранее не существует.

2.3 Ускорение 1: KV-кэш

Заметьте неэффективность наивного цикла: на каждом шаге модель заново обрабатывает **весь** текст с самого начала, хотя добавился только один токен. Внутри модели есть механизм «внимания» (attention), который для каждого токена вычисляет промежуточные значения — так называемые ключи (Keys) и значения (Values). Для старых токенов они не меняются — так зачем считать их заново?

KV-кэш (KV cache) — приём: промежуточные значения K и V для уже обработанных токенов сохраняются в памяти и переиспользуются. На каждом шаге по-настоящему обрабатывается только один новый токен.

Аналогия. Вы складываете длинный столбик чисел и после каждого нового числа... пересчитываете весь столбик с нуля. KV-кэш — это «запомнить промежуточную сумму» и просто прибавлять к ней новое число.

Эффект измерен: на компьютере Mac Mini M4 (CPU) скорость выросла с ~5 токенов в секунду до ~29 токенов в секунду — почти в 6 раз, без какого-либо изменения качества ответов. Результат побайтово тот же, только быстрее.

2.4 Ускорение 2: компиляция модели

Второй приём — `torch.compile`: PyTorch заранее «компилирует» вычисления модели в более эффективный машинный код (вместо того чтобы интерпретировать их шаг за шагом). Первый запуск медленный (идёт компиляция), зато потом — быстрее. Вместе с KV-кэшем на том же Mac Mini получилось ~68 токенов в секунду. Итого две чисто инженерные оптимизации дали ускорение более чем в 10 раз.

Режим (Mac Mini M4, CPU)	Токенов/сек
Наивная генерация	5
+ KV-кэш	29

+ KV-кэш + компиляция	68

Зачем это в книге про рассуждения? Потому что рассуждающие модели генерируют очень много токенов, и без этих ускорений эксперименты последующих глав заняли бы недели.

> **Ключевая мысль главы 2.** Текст генерируется строго по одному токену за шаг: предсказали → приклеили → повторили, пока не выпал токен-стоп. KV-кэш (не пересчитывать старое) и компиляция (оптимизировать вычисления) ускоряют этот цикл на порядок, не меняя результат.

Вопросы к главе 2

1. Опишите по шагам цикл генерации текста. Сколько токенов модель выдаёт за один проход?
2. Что такое EOS-токен и что произойдёт, если не останавливать генерацию при его появлении?
3. Какую лишнюю работу выполняет наивный цикл генерации и как её устраняет KV-кэш?
4. Меняет ли KV-кэш содержание ответов модели? А скорость — насколько (примерно) в эксперименте книги?
5. Почему для книги о рассуждающих моделях так важна скорость генерации?

Глава 3. Как измерить «ум» модели: оценка на математике

Прежде чем улучшать модель, нужен **измеритель**: иначе как понять, помогло улучшение или нет? Эта глава строит автоматическую систему проверки решений математических задач.

3.1 Почему математика

Математические задачи идеальны для проверки рассуждений: у них есть **один проверяемый правильный ответ**. Не нужен человек-оценщик — программа сама сравнит ответ модели с эталоном. Это свойство станет критически важным в главе 6 (там на нём построено обучение).

> **MATH-500** — стандартный набор из 500 математических задач разной сложности (алгебра, геометрия, теория чисел...) с известными правильными ответами. Это наш «экзаменационный билет»: прогоняем модель через все 500 задач и считаем процент правильных ответов — **точность (accuracy)**.

3.2 Конвейер проверки: от ответа модели до оценки

Казалось бы, просто: сравнить ответ модели с правильным. На деле модель пишет длинный текст со всякими рассуждениями, и из него надо аккуратно **выудить** финальный ответ. Конвейер (verifier — «верификатор») состоит из шагов:

1. **Генерация.** Модель получает задачу и просьбу записать финальный ответ в специальной «коробочке» — конструкции `\boxed{...}` (это обозначение из LaTeX, языка математической записи; например `\boxed{83}`).
2. **Извлечение.** Из ответа достаётся содержимое последней «коробочки». Если коробочки нет — запасной вариант: взять последнее число в тексте (или засчитать ошибку — зависит от настройки).
3. **Нормализация.** Ответ приводится к каноническому виду: убираются пробелы, доллары, единицы измерения; «0.5», «1/2» и «\frac{1}{2}» должны считаться одним и тем же числом.
4. **Проверка эквивалентности.** Нормализованный ответ модели сравнивается с эталонным — не как строки, а математически (чтобы «2/4» совпало с «1/2»).
5. **Оценка.** Совпало — 1 балл, нет — 0. Точность = сумма баллов / число задач.

Аналогия. Это автоматическая проверка ЕГЭ: ученик обязан вписать ответ в бланк строго в отведённую клетку (наша «коробочка»), сканер читает клетку, приводит запись к стандартному виду и сравнивает с ключом. Красивое решение на черновике сканер не волнует — только клетка с ответом.

В коде верификатора неожиданно много обработки ошибок — и это не перестраховка: на 500 реальных ответах модель чего только не выдаёт (кривой LaTeX, незакрытые скобки, ответ без коробочки). Урок: автоматическая оценка LLM — это на 20% идея и на 80% аккуратная обработка «грязных» выходов.

3.3 Результаты: точка отсчёта

Прогон двух моделей через все 500 задач MATH-500 дал:

Модель	Точность	Время (на 500 задач)
Qwen3 0.6B базовая	15,2%	~10 минут
Qwen3 0.6B рассуждающая (официальная)	48,2%	~182 минуты

Два вывода. Первый: рассуждения дают **троекратный** рост точности (15% → 48%) при той же самой архитектуре и размере модели — вся разница в пост-обучении. Второй: рассуждающая модель в ~18 раз медленнее, потому что пишет длиннющие цепочки рассуждений. Эти два числа — 15,2% и 48,2% — наши ориентиры на всю книгу: задача последующих глав — поднять базовые 15,2% как можно ближе к 48,2% (и выше) своими руками.

Ключевая мысль главы 3. Улучшать можно только то, что умеешь измерять. Мы построили автоматический верификатор: модель обязана дать ответ в «коробочке», программа извлекает его, нормализует и математически сравнивает с эталоном. Точка отсчёта: базовая модель — 15,2% на MATH-500, рассуждающая — 48,2%.

Вопросы к главе 3

1. Почему именно математические задачи удобны для автоматической оценки рассуждений?
2. Перечислите шаги конвейера проверки ответа — от текста, который выдала модель, до балла 0/1.
3. Зачем нужен шаг нормализации? Приведите пример двух записей одного и того же ответа.
4. Какие результаты показали базовая и рассуждающая версии Qwen3 0.6B на MATH-500 и чем объясняется разница во времени работы?
5. Зачем модель просят помещать ответ в `\boxed{...}`?

Глава 4. Путь 1: больше вычислений в момент ответа

Начинаем улучшать рассуждения. Первый путь — **масштабирование на инференсе** (inference-time scaling): модель не трогаем, но в момент ответа тратим больше вычислений, чтобы получить ответ точнее. В этой главе — два приёма: подсказка «думай по шагам» и голосование нескольких ответов.

4.1 Приём 1: Chain-of-thought промптинг

Самый дешёвый способ заставить модель рассуждать — **попросить её об этом**. К задаче добавляется фраза вроде «Рассуждай шаг за шагом» (think step by step). Базовая модель видела в обучающих текстах миллионы пошаговых решений, и такая подсказка «включает» этот стиль: модель начинает расписывать решение, и точность растёт.

> **Промпт (prompt)** — текст запроса, который подаётся модели. **CoT-промптинг** — добавление в промпт просьбы рассуждать по шагам.

Эффект впечатляет: одна строчка в промпте подняла точность базовой модели на MATH-500 с 15,2% до **40,6%**. Плата — время: ответы стали длиннее, и полный прогон занял 84 минуты вместо 10.

4.2 Подготовка к приёму 2: управляемая случайность

Второй приём потребует получать от модели **разные** ответы на один и тот же вопрос. Жадная генерация так не умеет (всегда один ответ). Нужно сэмплирование — и два регулятора для него.

Температура

> **Температура (temperature)** — число, на которое делят логиты перед превращением в вероятности. Температура < 1 «заостряет» распределение (вероятные токены становятся ещё вероятнее — ответы консервативнее и однообразнее). Температура > 1 «сглаживает» его (шансы выравниваются — ответы разнообразнее, но легче скатиться в чушь). Температура = 1 — ничего не меняется.

Аналогия. Температура — ручка «смелости» у импровизирующего музыканта. Прикрутили — играет только проверенные ноты, скучно, но без фальши. Выкрутили — смелые импровизации, иногда гениальные, чаще фальшивые.

Top-p

Проблема высокой температуры: ненулевой шанс получают и совсем нелепые токены. Решение — перед розыгрышем отсечь «хвост» распределения.

> **Top-p сэмплирование (nucleus sampling)** — выбор только из самых вероятных токенов, чьи вероятности в сумме дают p (например, 0.9 = 90%), остальные токены исключаются полностью. Затем среди оставшихся разыгрывается случайный выбор. Итог: разнообразие — есть, явный бред — отрезан.

4.3 Приём 2: самосогласованность (self-consistency)

Теперь главный приём главы. Идея: **спросить модель несколько раз и устроить голосование**.

1. Генерируем на одну задачу N разных решений (сэмплирование с температурой и top-p даёт каждый раз новую цепочку рассуждений).
2. Из каждого решения извлекаем финальный ответ (верификатором из главы 3).
3. Выбираем ответ, который встретился **чаще всего**.

Почему это работает: к правильному ответу ведёт много разных путей рассуждения, а ошибки обычно случайны и разнонаправлены. Разные цепочки, сходящиеся к одному ответу, — признак того, что ответ надёжен. Ошибки «голосуют» вразнобой, правильный ответ — хором.

Аналогия. Попросите пятерых школьников независимо решить задачу. Трое получили «83», один «38», один «90». Берём «83»: трое вряд ли ошиблись одинаково, идя разными путями.

Результаты на MATH-500 (базовая модель, если не указано иное)

Метод	Точность	Время
База, жадная генерация (глава 3)	15,2%	10 мин
CoT-промптинг	40,6%	84 мин
CoT + сэмплирование + голосование из 5	48,0%	453 мин
CoT + сэмплирование + голосование из 10	**52,0%**	863 мин
Для сравнения: официальная рассуждающая модель	48,2%	182 мин

Главный итог: комбинация «думай по шагам + 10 попыток + голосование» подняла **базовую** модель до 52% — выше официальной рассуждающей модели (48,2%)! Без единой минуты обучения. Плата — чудовищное время: 863 минуты против 182. В этом суть inference-time scaling: **меняем вычисления на точность**, ручка крутится в обе стороны.

> **Ключевая мысль главы 4.** Не меняя модель, точность можно радикально поднять: (1) попросить рассуждать по шагам; (2) сгенерировать несколько разнообразных решений (температура + top-p) и выбрать самый частый ответ. Чем больше попыток, тем точнее — и тем дороже.

Вопросы к главе 4

1. Что такое inference-time scaling? Чем этот путь принципиально отличается от обучения с подкреплением и дистилляции?
2. Как работает CoT-промптинг и почему простая фраза в запросе так сильно влияет на точность?
3. Что делает температура с распределением вероятностей при значениях меньше и больше единицы?
4. Зачем нужен top-p, если уже есть температура? Какую проблему он решает?
5. Опишите алгоритм самосогласованности. Почему голосование чаще выбирает правильный ответ, а не популярную ошибку?

Глава 5. Путь 1 (продолжение): оценка уверенности и самоулучшение

В главе 4 мы выбирали ответ голосованием. Но голосование грубовато: а если голоса разделились поровну? А можно ли **оценить качество** каждого ответа числом и выбрать лучший? Эта глава строит «оценщики» ответов и на их основе — цикл самоулучшения.

5.1 Оценщик 1: простая эвристика

Самый примитивный способ оценить ответ — по внешним признакам: например, по длине (слишком короткий ответ — вероятно, модель не рассуждала; чрезмерно длинный — возможно, зациклилась) и по наличию «коробочки» с финальным ответом. Это называется **эвристический скоринг** — быстрые грубые правила. Работает так себе, но бесплатно; годится как разрешитель ничьих при голосовании.

5.2 Оценщик 2: уверенность самой модели

Идея тоньше: модель сама «знает», насколько она уверена в каждом сгенерированном токене — это и есть вероятность, с которой она его выбрала! Если собрать эти вероятности по всему ответу, получится численная **мера уверенности модели в собственном ответе**.

Проблема перемножения

Вероятность всей последовательности = произведение вероятностей её токенов (вероятность первого токена × вероятность второго при уже известном первом × ...). Но вероятности — числа меньше единицы, и при перемножении сотен таких чисел результат становится настолько микроскопическим (10 в минус сотых степенях), что компьютер теряет точность и округляет его до нуля. Это называется «числовое переполнение вниз» (underflow).

Решение: логарифмы

> **Лог-вероятность (log-probability, logprob)** — логарифм вероятности. Главное свойство логарифма: он превращает умножение в сложение. Вместо перемножения тысячи крошечных чисел — складываем тысячу «нормальных» отрицательных чисел (например, −0.1, −2.3). Никакого переполнения. Чем ближе logprob к нулю, тем увереннее модель (вероятность 1 → logprob 0; вероятность 0.01 → logprob ≈ −4.6).

Аналогия. Логарифмы — как децибелы для громкости: реальные значения различаются в миллиарды раз, а на шкале децибел — удобные числа 30, 60, 90, и «умножение громкостей» превращается в сложение децибел.

Итоговая формула уверенности

Оценка уверенности ответа = **средняя лог-вероятность по токенам ответа** (только ответа, без промпта). Почему средняя, а не сумма? Сумма штрафует длинные ответы (каждый токен добавляет отрицательное число), а среднее делает ответы разной длины сравнимыми. Запомните этот нюанс «сумма или среднее» — он выстрелит в главе 6.

5.3 Применение оценщиков: Best-of-N

> **Best-of-N** — стратегия: генерируем N вариантов ответа, оцениваем каждый (например, уверенностью), берём ответ с лучшей оценкой. Отличие от самосогласованности: там побеждает самый частый ответ, здесь — самый качественный по оценке (даже если он один такой).

5.4 Самоулучшение (self-refinement)

Второй большой приём главы: использовать модель **против самой себя** — просить её улучшать собственные ответы.

Цикл самоулучшения:

1. Модель генерирует первый вариант решения.
2. Модели показывают её же решение и просят: «Проверь это решение, найди ошибки и выдай улучшенную версию».
3. Новый вариант оценивается (например, уверенностью по logprobs). Стало лучше — берём его; стало хуже — оставляем старый.
4. Повторяем несколько итераций.

Аналогия. Черновик → перечитал свежим взглядом → исправил → ещё раз перечитал. Автор и редактор в одном лице. Важно: «редактор» здесь не умнее «автора» — это та же модель, поэтому чудес ждать не стоит: она нередко не видит собственных ошибок или «исправляет» правильное на неправильное. Поэтому и нужен шаг 3 — внешняя оценка, чтобы не принимать ухудшения.

Honest-вывод книги: у маленькой модели самоулучшение помогает скромно и нестабильно — заметно слабее, чем голосование из главы 4. Но сам приём важен

концептуально: это шаг к системам, где модель итеративно работает над задачей, а не отвечает одним выстрелом.

> **Ключевая мысль главы 5.** Ответ модели можно оценить числом — средней лог-вероятностью его токенов («уверенность»). Логарифмы нужны, чтобы произведение крошечных вероятностей не обнулилось: умножение превращается в сложение. С оценщиком в руках доступны стратегии Best-of-N (выбрать лучший из N) и self-refinement (итеративно улучшать свой ответ, принимая только улучшения).

Вопросы к главе 5

1. Как вычислить вероятность целой сгенерированной последовательности из вероятностей отдельных токенов? Какая техническая проблема при этом возникает?
2. Что такое лог-вероятности и какое свойство логарифма спасает от этой проблемы?
3. Почему для оценки уверенности ответа берут среднюю, а не суммарную лог-вероятность?
4. Чем Best-of-N отличается от самосогласованности из главы 4?
5. Опишите цикл самоулучшения. Зачем в нём внешняя оценка (шаг «принять или откатить») и почему без неё цикл может ухудшать ответы?

Глава 6. Путь 2: учим модель рассуждать через подкрепление (GRPO)

Самая важная глава книги. До сих пор модель не менялась — мы лишь хитро её использовали. Теперь мы будем **дообучать** модель, чтобы она сама, без подсказок и голосований, научилась рассуждать. Именно так был создан DeepSeek-R1. Глава плотная — читайте медленно.

6.1 Идея обучения с подкреплением

> **Обучение с подкреплением (Reinforcement Learning, RL)** — способ обучения, при котором модель не показывают правильные решения, а дают ей **пробовать самой** и выдают **награду** (reward) за хорошие результаты. Модель постепенно меняется так, чтобы получать больше наград. В терминологии RL обучаемую модель называют **политикой** (policy).

> **Аналогия.** Дрессировка: собаке не объясняют теорию команды «сидеть» — она пробует разные действия, за правильное получает лакомство, и нужное поведение закрепляется.

RLHF: как учили ChatGPT

Классическая схема RL для LLM — **RLHF** (RL from Human Feedback, «с обратной связью от людей»): люди сравнивают пары ответов модели («этот лучше того»), на этих сравнениях обучают отдельную **модель-награду**, и уже она оценивает ответы при обучении. Так GPT-3 превратили в ChatGPT. Минусы: дорогая человеческая разметка и отдельная большая модель-награда.

RLVR: прорыв DeepSeek-R1

> **RLVR** (RL with Verifiable Rewards — «RL с проверяемыми наградами») — замена человеческих оценок **автоматической проверкой**: для математики — верификатор из главы 3 (ответ сошёлся → награда 1, нет → 0), для кода — компиляция и тесты. Не нужны ни люди, ни модель-награда. Именно RLVR позволил DeepSeek-R1 в 2025 году добиться сильных рассуждений — и произвёл фурор.

Поразительный факт из DeepSeek-R1: модель никто не учил писать длинные цепочки рассуждений. Её награждали **только за правильный финальный ответ** — и длинные рассуждения, самопроверки, «подожди-ка, проверю ещё раз» **возникли**

сами, потому что они статистически ведут к правильным ответам. Рассуждение — выученная стратегия максимизации награды.

6.2 Алгоритм GRPO: общая картина

Для обновления модели нужен конкретный алгоритм. Классика — PPO, но он требует ещё одну нейросеть (оценщик «ценности»). DeepSeek использовал более простой и экономный **GRPO**.

> **GRPO (Group Relative Policy Optimization)** — «групповая относительная оптимизация политики». Ключевая идея в названии: на каждую задачу генерируется **группа** из нескольких ответов, и качество каждого оценивается **относительно остальных в группе**. Не нужна отдельная модель-оценщик — группа сама себе система координат.

Аналогия (из книги — «шеф-повар»). Повар готовит одно блюдо 4 раза чуть по-разному (группа попыток). Дегустатор ставит оценки. Повар сравнивает попытки **между собой**: что общего у удачных? что я делал в неудачных? — и сдвигает свою манеру готовки в сторону удачных приёмов. Никакого «идеального рецепта» ему не выдают.

Шаги одного цикла GRPO

1. **Роллауты.** Берём задачу из тренировочного набора (12 000 задач MATH, не пересекающихся с экзаменационным MATH-500). Модель генерирует группу из N ответов (например, 8) с сэмплированием — ответы получаются разные. Каждый сгенерированный ответ называется **роллаут** (rollout).
2. **Награды.** Верификатор из главы 3 выдаёт каждому роллауту награду: 1.0 — финальный ответ правильный И оформлен в коробочке `\boxed{}`, иначе 0.0. (Требование коробочки — скрытая «награда за формат».)
3. **Преимущества.** Награды переводятся в **преимущества** (advantages) — центральное понятие GRPO, см. ниже.
4. **Лог-вероятности.** Для каждого роллаута считаем его **суммарную** лог-вероятность (знакомая механика из главы 5 — но теперь сумма, а не среднее: сигнал обучения должен отражать всю последовательность целиком).
5. **Функция потерь и обновление.** Собираем всё в одно число (loss) и делаем шаг обновления весов модели: повысить вероятности хороших роллаутов, понизить — плохих.

6.3 Преимущества: сердце GRPO

Формула простая: из награды каждого роллаута вычитаем **среднюю награду группы** и делим на разброс (стандартное отклонение) группы:

```
преимущество_i = (награда_i − средняя_награда_группы) / разброс_группы
```

Что это даёт. Пусть из 4 роллаутов правильный один (награды 1, 0, 0, 0; средняя 0.25). Тогда правильный получает большое **положительное** преимущество (он лучше среднего по группе), а три неправильных — **отрицательное** (хуже среднего). Модель усилит то, что делала в правильном, и ослабит то, что в неправильных.

Важное следствие: если **все** ответы группы правильные или **все** неправильные, у всех награда равна средней, все преимущества равны нулю — **обучающего сигнала нет**, модель не обновляется на этой задаче. Учёба происходит только там, где внутри группы есть контраст между успехом и неудачей: задачи «на границе возможностей» модели. Слишком лёгкое и слишком трудное ничему не учит.

> **Простыми словами:** оценивание «по кривой», относительно одногруппников. «Молодец ли этот ответ?» определяется не абсолютно, а сравнением с другими попытками на ту же задачу. А если вся группа сдала на отлично (или вся завалила) — сравнивать нечего, урок отменяется.

6.4 Функция потерь: как сигнал попадает в веса

Итоговая формула (понимать её дословно не обязательно, важен смысл): потеря = −среднее по роллаутам (преимущество × суммарная лог-вероятность роллаута). Оптимизатор PyTorch минимизирует потерю, а значит (из-за минуса) **максимизирует** лог-вероятности роллаутов с положительным преимуществом и снижает — с отрицательным. По-русски: «делай чаще то, что сработало лучше одногруппников, и реже то, что сработало хуже».

6.5 Результаты обучения

Автор запускает цикл GRPO на Qwen3 0.6B base: 8 роллаутов на задачу, до 512 токенов на ответ, всего 50 шагов обучения (это очень мало, но GRPO прожорлив).

Модель	MATH-500	Средняя длина ответа (токены)
База (до обучения)	15,2%	79
После 50 шагов GRPO	**47,4%**	586
Официальная рассуждающая (ориентир)	48,2%	1370

За какие-то 50 шагов самодельное RL-обучение почти догнало официальную рассуждающую модель (47,4% против 48,2%)! Обратите внимание на длину ответов: 79 → 586 токенов. Модель **сама** начала писать длинные рассуждения — её за это не награждали напрямую, только за правильные ответы. Предупреждение автора: дольше — не значит лучше; ванильный GRPO нестабилен, при длительном обучении модель может деградировать. Лечение — в главе 7.

> **Ключевая мысль главы 6.** RLVR = модель решает задачи, автоматический верификатор раздаёт награды за правильные ответы. GRPO превращает награды в обучающий сигнал через сравнение внутри группы ответов: лучше среднего — усилить, хуже — ослабить; все одинаковые — сигнала нет. Награждая только за правильность, получаем модель, которая сама научилась рассуждать: 15,2% → 47,4% за 50 шагов.

Вопросы к главе 6

1. Чем RLVR отличается от RLHF и почему для математики RLVR особенно удобен?
2. Что такое роллаут и зачем GRPO генерирует несколько роллаутов на одну задачу, а не один?
3. Как из наград вычисляются преимущества и что произойдёт с обучением на задаче, где все 8 роллаутов оказались правильными?
4. За что именно модель получала награду при обучении — и откуда тогда взялись длинные цепочки рассуждений (ответы выросли с 79 до 586 токенов)?
5. Каких результатов достигло GRPO-обучение за 50 шагов по сравнению с базовой и официальной рассуждающей моделью?

Глава 7. Путь 2 (продолжение): стабилизируем и улучшаем GRPO

Глава 6 дала работающий, но «сырой» алгоритм: обучение нестабильно и со временем может разваливаться. Глава 7 — про то, как следить за здоровьем обучения и какие «предохранители» добавить. Это самая техническая глава книги (автор прямо пишет, что её можно пропустить); мы возьмём из неё главное.

7.1 Приборная панель: за чем следить во время обучения

RL-обучение легко «тихо ломается», поэтому нужны метрики — графики, по которым видно состояние процесса:

- **Средняя награда.** Должна расти: модель решает всё больше задач. Коварный нюанс: награда 1.0 у всех — это одновременно «отлично» и «обучение остановилось» (вспомните главу 6: без контраста нет сигнала). Пора брать задачи посложнее или останавливаться.
- **Длина ответов.** В начале должна расти (модель осваивает рассуждения). Резкое падение или взрывной рост — тревожные знаки.
- **Точность на внешнем экзамене (MATH-500).** Главный показатель. В эксперименте книги она сначала росла, а после ~шага 400 начала падать — то самое «разваливание» ванильного GRPO.
- **Loss.** В отличие от обычного обучения, здесь само число loss малоинформативно — важнее остальные метрики.
- **Разброс преимуществ (advantage std).** Среднее преимуществ всегда 0 (так устроена формула — это просто проверка на баги). А вот разброс показывает силу обучающего сигнала: около 1 — здоровый сигнал; упал к 0 — роллауты стали одинаковыми, учиться не на чем.
- **Энтропия** — мера «разброса внимания» модели при выборе следующего токена. Низкая — модель почти детерминирована (риск зацикливания на однотипных ответах, «коллапс»); умеренная — норма; очень высокая — модель близка к случайному бреду.

Аналогия. Это приборы в кабине самолёта: ни один прибор в отдельности не говорит «всё хорошо», но вместе (награда растёт, разброс преимуществ жив, энтропия умеренная, экзамен улучшается) они дают картину полёта. Опытный пилот смотрит на все сразу.

7.2 Предохранитель 1: обрезка шага обновления (clipping)

Проблема: иногда один шаг обучения слишком резко меняет модель — и всё наработанное разваливается. Решение, позаимствованное из алгоритма PPO: **ограничить размер изменения**.

Механика: после обновления сравнивают, насколько новая версия модели оценивает сгенерированные ответы иначе, чем старая (отношение их вероятностей — **policy ratio**; 1.0 = ничего не изменилось). Если отношение выходит за коридор «1 ± eps» — его принудительно **обрезают** (clip) до границы коридора. Слишком резкие обновления смягчаются.

> **Аналогия.** Ограничитель руля: как бы резко водитель ни дёрнул, колёса повернутся не больше чем на разрешённый угол. Медленнее входим в повороты — зато не вылетаем с трассы.

Результат в эксперименте книги: с обрезкой обучение прошло стабильно — провал около шага 400 исчез.

7.3 Предохранитель 2: KL-штраф — и почему его часто выключают

Второй классический предохранитель — **KL-штраф**: к функции потерь добавляется наказание за то, что обучаемая модель слишком далеко «уехала» от исходной (эталонной копии, сделанной до начала обучения). KL-дивергенция — мера расхождения между двумя распределениями вероятностей. Смысл: «улучшайся, но не теряй себя» — не забывай язык, знания и здравый смысл исходной модели.

Неожиданный поворот: в эксперименте книги добавление KL-штрафа всё **сломало** — после ~200 шагов награды обнулились, и KL-штраф, оставшись единственным действующим сигналом, растащил модель до генерации случайного бреда (точность 0%). Вывод, совпадающий с практикой современных работ (Dr. GRPO, Olmo 3, DeepSeek V3.2): **при обучении на математике KL-штраф часто лучше отключить** — меньше сложности, меньше памяти (не нужно держать копию эталонной модели), стабильнее результат. Урок шире: в RL «стандартный компонент» ≠ «обязательный компонент», всё проверяется экспериментом.

7.4 Награда за формат и токены <think>

Многие рассуждающие модели (DeepSeek-R1, Qwen3) оборачивают рассуждения в специальные теги: `<think>` ...цепочка рассуждений... `</think>`, а затем финальный ответ. Это удобно: интерфейс может скрывать «мысли» от пользователя, показывая только ответ.

Чтобы приучить модель к такому формату, вводят **вторую награду — за формат**: +1, если в ответе есть оба тега в правильном порядке. Итоговая награда = награда за правильность + вес × награда за формат.

Два поучительных факта из экспериментов:

- Базовую модель так учить бессмысленно: она никогда не видела тег `<think>` (его даже нет в её словаре целым токеном), поэтому формату сначала учат «мягким» дообучением на примерах, а RL — потом.
- Если награда за формат слишком весома, модель начинает «фармить» лёгкие баллы за теги и перестаёт стараться решать правильно — качество падает. Лечение: снизить вес формата (например, до 0.1) или давать его только при правильном ответе.

Простыми словами: это закон Гудхарта — «когда метрика становится целью, она перестаёт быть хорошей метрикой». Модель оптимизирует ровно то, за что платят, а не то, что вы имели в виду. Проектирование наград — отдельное искусство.

Ключевая мысль главы 7. RL-обучение требует мониторинга (награда, длина ответов, разброс преимуществ, энтропия, внешний экзамен) и предохранителей. Обрезка шага обновления (clipping) стабилизирует обучение; KL-штраф в теории удерживает модель от деградации, но на математике часто вредит, и его отключают. Награды надо проектировать осторожно: модель оптимизирует именно то, за что награждают.

Вопросы к главе 7

1. Почему средняя награда 1.0 — это одновременно и успех, и проблема? Что в этот момент происходит с обучающим сигналом?
2. Что показывает разброс (std) преимуществ и о чём говорит его падение к нулю?
3. Как работает обрезка (clipping) шага обновления и от чего она защищает?
4. Зачем в GRPO придуман KL-штраф и почему в экспериментах книги его в итоге отключили?
5. Что пошло не так, когда модель стали награждать за теги <think>, и какими двумя способами это лечится? Какой общий урок о проектировании наград отсюда следует?

Глава 8. Путь 3: дистилляция — маленький учится у большого

Финальный метод. RLVR из глав 6–7 — это «учиться на собственных пробах и ошибках». Дистилляция — «учиться у мастера».

8.1 Идея

> **Дистилляция (distillation)** — обучение маленькой модели (**ученика**, student) на выходах большой умной модели (**учителя**, teacher). Знания большой модели как бы «перегоняются» в маленькую.

Зачем? Большая рассуждающая модель (например, DeepSeek-R1 на 671 млрд параметров) слишком дорога для повседневной работы. Маленькая модель дешёвая, но глупая. Дистилляция — способ получить маленькую и при этом неплохо рассуждающую модель. Факт из статьи DeepSeek-R1: для малых моделей дистилляция от R1 дала результаты **лучше**, чем попытка обучить те же малые модели напрямую через RLVR. Маленькой модели трудно «открыть» рассуждения самой, но легко перенять готовые.

8.2 Жёсткая и мягкая дистилляция

	Жёсткая (hard)	Мягкая (soft)
Что берём у учителя	Только готовый текст ответов	Полное распределение вероятностей по всем токенам словаря на каждом шаге
Чему учится ученик	Воспроизводить тексты учителя (обычное обучение «предскажи следующий токен», но на текстах учителя)	Подражать всей «манере думать»: повторять вероятности учителя
Требования	Любой доступ к учителю, хоть через интернет-API	Доступ к внутренностям учителя + одинаковый токенизатор у обоих
На практике	**Стандарт** (так делал DeepSeek-R1)	Редкость (логиты обычно недоступны, хранить их дорого)

Аналогия. Жёсткая дистилляция — учиться готовить по готовым блюдам шефа: пробуешь и воспроизводишь. Мягкая — стоять у шефа за плечом и перенимать каждое микродвижение: какие варианты он рассматривал, почему отверг. Второе глубже, но шеф должен пустить на кухню — а DeepSeek или OpenAI «на кухню» (к логитам) не пускают.

8.3 Как это сделано в книге

1. **Сбор данных учителя.** Те же 12 000 задач MATH (что и в главах 6–7) отправили через облачный API большому DeepSeek-R1 (671B). Он выдал на каждую задачу цепочку рассуждений в тегах `<think>` и финальный ответ. Стоимость — около $50 за API. Датасет сгенерирован автором заранее и выложен — читателю платить не нужно.
2. **Подготовка примеров.** Каждый пример склеивается: промпт (задача) + ответ учителя (рассуждение + финал), размечается спецтокенами диалога, токенизируется. Используется токенизатор рассуждающего варианта Qwen3 — он «знает» теги `<think>` как отдельные токены.
3. **Обучение.** Ученик (наш Qwen3 0.6B base) учится предсказывать токены ответов учителя. Функция потерь — классическая **кросс-энтропия**: по сути те же лог-вероятности из главы 5 — штраф тем больше, чем меньшую вероятность ученик дал «правильному» (учительскому) токену. Технически это обычное **дообучение с учителем** (supervised fine-tuning) на синтетических данных — никаких наград, роллаутов и преимуществ. За процессом следят по **validation loss** — ошибке на отложенных примерах, которые модель не видела при обучении: она должна снижаться.

8.4 Результаты и мораль

Модель / данные учителя	MATH-500
База (до всего)	15,2%
Дистилляция от DeepSeek-R1 (3 эпохи)	33,6%
Дистилляция от Qwen3 235B (3 эпохи)	**45,0%** (лучший результат на 1-й эпохе)
GRPO из главы 6 (для сравнения)	47,4%
Официальная рассуждающая (ориентир)	48,2%

Интересное наблюдение: учитель Qwen3 235B дал ученику заметно больше, чем более знаменитый DeepSeek-R1 (45% против 33,6%). Вероятная причина: учитель из **той же семьи** моделей — его стиль и формат ближе и «усвояемее» для ученика. Мораль: качество и совместимость данных учителя важнее громкого имени.

8.5 Итоговая карта книги

Метод	Обучение?	MATH-500	Главный ресурс
Базовая модель, как есть	—	15,2%	—
CoT-промптинг (гл. 4)	Нет	40,6%	Время на инференсе
CoT + голосование из 10 (гл. 4)	Нет	52,0%	Очень много времени на инференсе (каждый раз!)
RLVR + GRPO, 50 шагов (гл. 6)	Да	47,4%	Дорогое обучение — зато один раз
Дистилляция от Qwen3 235B (гл. 8)	Да	45,0%	Доступ к учителю + дешёвое обучение

И финальная мысль: пути **комбинируются**. Реальные системы (тот же DeepSeek-R1) используют всё сразу: дистилляцию и дообучение на примерах — чтобы задать формат и базу, RL — чтобы отточить рассуждения, а приёмы инференса (голосование, больше «времени на подумать») — чтобы выжать максимум в момент ответа.

> **Ключевая мысль главы 8.** Дистилляция — перенос умений большой модели в маленькую через обучение на её ответах. На практике почти всегда «жёсткая»: собрали ответы учителя → дообучили ученика предсказывать их (обычная кросс-энтропия, никакого RL). Для маленьких моделей это часто эффективнее собственного RLVR, а выбор учителя (совместимость с учеником) решает больше, чем его знаменитость.

Вопросы к главе 8

1. Кто такие «учитель» и «ученик» в дистилляции и какую проблему дистилляция решает?
2. Чем жёсткая дистилляция отличается от мягкой и почему на практике почти всегда используют жёсткую?
3. Опишите три шага дистилляции, проделанные в книге. Чем это обучение технически отличается от GRPO из главы 6?
4. Почему учитель Qwen3 235B оказался для ученика полезнее знаменитого DeepSeek-R1? Какой вывод из этого следует?
5. Сравните три пути книги (инференс-приёмы, RLVR, дистилляция): когда какой выгоднее? Почему на практике их комбинируют?

Заключение и что дальше

Пройдём маршрут ещё раз, одним абзацем. LLM — предсказатель следующего токена (гл. 0). Рассуждение — это генерация промежуточных шагов, резко повышающая точность (гл. 1). Мы научились генерировать текст и ускорять генерацию (гл. 2), построили честный измеритель — верификатор на MATH-500 (гл. 3). Затем три пути к рассуждениям: выжать больше на инференсе — просьба думать по шагам, голосование, оценка уверенности, самоулучшение (гл. 4–5); дообучить модель через награды за правильные ответы — RLVR с алгоритмом GRPO и его предохранителями (гл. 6–7); перенять умения большой модели — дистилляция (гл. 8).

Если после проверки всех ответов захотите углубиться:

- Репозиторий книги: **github.com/rasbt/reasoning-from-scratch** — все ноутбуки с кодом по главам; приложения C–G (архитектура Qwen3, большие модели, батчинг, методы оценки LLM, чат-интерфейс).
- Предыдущая книга автора «Build a Large Language Model (From Scratch)» — если захочется понять, как устроена сама LLM изнутри (трансформер, внимание, предобучение).
- Блог автора: magazine.sebastianraschka.com — регулярные разборы свежих методов RL для LLM.

Напоминание о порядке работы: отвечайте на вопросы после каждой главы и присылайте ответы в чат в формате «Глава N: 1) ... 2) ... 3) ... 4) ... 5) ...». Я проверю каждый ответ, укажу на неточности и при необходимости объясню тему другими словами. Удачи!